FaceAuth: Single-Image Synthetic Data Generation and MobileNetV2-Based Binary Face Authentication on Android with Filesystem Lockdown

Pranay Dadi
CS402M — Computer Systems Security
cs23b013@iittp.ac.in

Bandaru Charan Teja
CS402M — Computer Systems Security
cs23b011@iittp.ac.in

Abstract

We present **FaceAuth**, a two-class face authentication system trained exclusively from two single reference photographs — one per identity — and deployed as an Android application. Starting from one image per class, we generate 500 synthetic variations each through geometric pose simulation, photometric illumination transfer, and scale perturbation, then apply conventional augmentation to reach 500 images per class with an 80/10/10 train/val/test split. A MobileNetV2 backbone pretrained on ImageNet is fine-tuned using a two-phase transfer learning protocol, achieving **100% accuracy** on both classes of the held-out test set and a ROC-AUC of 1.0. The trained model is exported to TensorFlow Lite and embedded in an Android application with two authentication paths: Class A (authenticated user) triggers an access-granted screen with a timestamped log, while Class B detection above a confidence threshold of 0.85 triggers an access-denied screen with a live filesystem lockdown demonstration proving Android's sandboxed security model prevents unauthorised access. All code, dataset, and model weights are released publicly with tag `v1.0-dataset`.

1 Introduction

Face authentication is a ubiquitous access-control mechanism, yet its security depends critically on the quality and diversity of the training data. Real-world deployment often reduces to an adversarial scenario in which an attacker possesses a single photograph of a target and must determine whether a classifier trained from that single image can be fooled or, conversely, whether a defender can build a reliable classifier from the same sparse data.

This work addresses both sides of that question within the framework of CS402M (Computer Systems Security). Our specific contributions are:

1. A **single-image generative pipeline** that produces 500+ diverse synthetic face variations per class from one photograph, spanning pose, illumination, and scale.
2. A **two-phase MobileNetV2 transfer-learning protocol** trained on the resulting dataset with full hyperparameter documentation.
3. An **Android authentication app** with two clearly distinct authentication paths and a real-time filesystem lockdown demonstration.
4. A **public dataset release** (GitHub tag `v1.0-dataset`) with complete reproducibility documentation.
5. A **critical analysis** of the failure modes inherent in synthetic-data training and the mitigation strategies applied.

The remainder of this paper is structured as follows. Section 2 surveys related work. Section 3 describes the full pipeline. Section 4 presents quantitative and qualitative results. Section 5 discusses corner cases, limitations, and distributional shift. Section 6 concludes.

2 Related Work

2.1 Few-shot and single-shot face recognition

Classical face recognition pipelines — Eigenfaces **?**, FisherFaces, and Local Binary Patterns — require multiple enrolment images per subject. Deep metric-learning approaches such as FaceNet **?** and ArcFace **?** substantially reduce this requirement by embedding faces in a compact Euclidean space, enabling one-shot recognition. Our work is complementary: rather than modifying the loss function, we expand the training set synthetically from a single image.

2.2 Synthetic data augmentation

Generative data augmentation has been studied extensively in low-data regimes. Shorten and Khoshgoftaar **?** survey conventional augmentation techniques including flipping, rotation, colour jitter, and noise injection, finding consistent accuracy improvements across domains. More recently, diffusion-model fine-tuning (DreamBooth **?**) allows subject-specific generation from 3–5 reference images, producing photorealistic synthetic variations. Our pipeline occupies the middle ground: we apply deterministic geometric and photometric transforms that are fully reproducible and require no GPU-accelerated generative model, making the approach accessible without high-performance hardware.

2.3 Transfer learning for face classification

MobileNetV2 **?** is a lightweight convolutional architecture designed for mobile deployment. Its inverted residual structure and linear bottlenecks achieve competitive accuracy with a fraction of the parameters of VGG or ResNet. Howard et al. **?** demonstrate that ImageNet-pretrained MobileNet features transfer well to face tasks. We adopt MobileNetV2 over heavier architectures because the target deployment is an Android emulator with constrained RAM.

2.4 On-device face authentication

Google ML Kit provides on-device face detection with sub-30ms latency on mobile hardware **?**. TensorFlow Lite **?** enables execution of quantized neural networks on Android with negligible inference overhead. Prior work combining ML Kit face detection with a TFLite classifier has demonstrated real-time authentication pipelines; our contribution is the integration of a synthetic-data-trained model, a confidence-threshold gate, and a security-response (filesystem lockdown) module.

3 Methodology

3.1 Problem formulation

Given one reference photograph per class — Class A (authenticated user) and Class B (target/unauthorised) — we formulate authentication as a binary classification problem: given a camera frame $\mathbf{x} \in \mathbb{R}^{224\times224\times3}$, predict $y \in \{A, B\}$, where Class B prediction above threshold $\tau = 0.85$ triggers an intrusion response.

3.2 Dataset generation

3.2.1 Stage 1 — Geometric pose simulation

Starting from a single source image $I_0 \in \mathbb{R}^{224\times224\times3}$, we generate out-of-plane pose variations using a two-step transform:

In-plane rotation (roll): A 2D affine transform $\mathbf{M}_{\text{roll}} = \text{getRotationMatrix2D}(\mathbf{c}, \alpha_z, 1.0)$ rotates the image about its centre $\mathbf{c}$ by roll angle $\alpha_z \in [-10°, +10°]$.

Yaw/pitch simulation: A perspective warp $\mathbf{M}_{\text{persp}} = \text{getPerspectiveTransform}(\mathbf{P}_{\text{src}}, \mathbf{P}_{\text{dst}})$ displaces corner points proportionally to $\tan(\alpha_y) \times 0.28$ (yaw) and $\tan(\alpha_x) \times 0.18$ (pitch), simulating lateral and vertical head rotation without a 3D face model. Border regions are filled by reflection padding.

3.2.2 Stage 2 — Illumination simulation

Eight photometric modes are applied via PIL `ImageEnhance` and channel-wise arithmetic:

Table 1: Illumination simulation modes and their parameter ranges.

Mode	Operation
`bright`	Brightness $\times[1.2, 1.8]$
`dark`	Brightness $\times[0.3, 0.7]$
`contrast`	Contrast $\times[0.4, 2.0]$
`warm`	$R \mathrel{+}= [20, 40]$; $B \mathrel{-}= [15, 30]$
`cool`	$B \mathrel{+}= [20, 40]$; $R \mathrel{-}= [15, 30]$
`shadow`	Directional linear gradient mask, strength $[0.3, 0.6]$
`overexposed`	High brightness + low contrast
`flat`	Identity (no change)

3.2.3 Stage 3 — Scale and compression variation

Scale factors $s \in [0.75, 1.30]$ are applied via bilinear resize with centre-crop (upscale) or reflection padding (downscale). JPEG re-encoding at quality $q \in [60, 95]$ introduces compression artefacts. Gaussian blur with kernel size $\in \{3, 5\}$ pixels simulates camera defocus.

3.2.4 Stage 4 — Systematic grid + random combinations

The first pass generates a systematic grid of $5 \times 5 \times 3 \times 8 = 600$ pose–lighting combinations. The second pass fills the remaining quota with random 1–4 operation compositions until $N = 500$ images per class are produced.

3.3 Conventional augmentation and dataset split

The 500 synthetic images per class are further augmented with the pipeline described in Table 2.

Table 2: Conventional augmentation hyperparameters.

Transform	Probability	Parameter range
Horizontal flip	0.50	—
Brightness jitter	0.70	factor $\in [0.60, 1.40]$
Rotation	0.60	angle $\in [-15°, +15°]$
Gaussian noise	0.50	$\sigma \in [5, 25]$
Contrast jitter	0.40	factor $\in [0.70, 1.50]$
Saturation jitter	0.30	factor $\in [0.70, 1.30]$

At least one augmentation is guaranteed per image. The dataset is split deterministically (seed = 42) into 80% train / 10% validation / 10% test using stratified sampling, yielding 400/50/50 images per class per split.

3.4 Model architecture

We adopt MobileNetV2 **?** pretrained on ImageNet as the feature extractor. The full model is:

$$f(\mathbf{x}) = \text{Softmax}(W_2 \cdot \text{GELU}(W_1 \cdot \text{Dropout}_{0.30}(\text{GAP}(\text{MobileNetV2}(\text{preprocess}(\mathbf{x} \cdot 255))))))$$

where GAP denotes global average pooling, $W_1 \in \mathbb{R}^{128\times1280}$ with $\ell_2 = 10^{-4}$ regularisation, and $W_2 \in \mathbb{R}^{2\times128}$. The preprocessing layer internally calls `mobilenet_v2.preprocess_input`, mapping $[0, 255]$ pixel values to $[-1, 1]$. The Android app normalises to $[0, 1]$ before passing to TFLite; the embedded preprocessing layer handles the remainder.

Total parameters: 2,422,210 (2,259,394 in base; 162,816 in head).

3.5 Two-phase training protocol

Phase 1 — Frozen base: MobileNetV2 weights are frozen. Only the classification head is trained for 10 epochs using Adam with learning rate $\eta = 10^{-3}$. Early stopping (patience = 5 on validation accuracy) prevents overfitting.

Phase 2 — Fine-tuning: Layers 100–154 of MobileNetV2 are unfrozen. The full model is trained for up to 20 additional epochs with $\eta = 10^{-5}$ and the same early-stopping schedule. A `ReduceLROnPlateau` callback halves the learning rate when validation loss plateaus for 3 epochs.

Table 3: Full hyperparameter specification.

Hyperparameter	**Value**
Base model	MobileNetV2 (ImageNet pretrained)
Input resolution	$224 \times 224 \times 3$
Head: Dense units	128, ReLU, $\ell_2 = 10^{-4}$
Head: Dropout	0.30 (after GAP), 0.15 (after Dense)
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Phase 1 LR	10^{-3}, batch 32, 10 epochs
Phase 2 LR	10^{-5}, batch 32, 20 epochs
Fine-tune from layer	100 / 154
Early stopping patience	5 (monitor: val_accuracy)
LR reduction factor	0.5, patience 3
Loss function	Categorical cross-entropy
Class B threshold τ	0.85
Random seed	42
TFLite quantization	Dynamic-range INT8

3.6 TFLite export and Android deployment

The best Phase-2 Keras model is converted to TFLite with dynamic-range INT8 quantisation using `tf.lite.Optimize.DEFAULT`. This reduces model size from 9.3 MB (float32) to 2.6 MB (INT8) with no measurable accuracy degradation on the test set. The quantised model is placed in the Android `assets/` directory and loaded via memory-mapped `FileChannel` at runtime.

3.7 Android application architecture

The app comprises four activities interconnected as shown below:

MainActivity → **CameraActivity** → {AuthSuccessActivity | AuthFailActivity}

CameraActivity implements the following per-frame pipeline:

1. CameraX `ImageAnalysis` captures YUV_420_888 frames at the device's native resolution.
2. Each frame is converted to an NV21 byte array and decoded to an RGB Bitmap via `YuvImage.compressToJpeg`.
3. ML Kit Face Detection (fast mode, minimum face size 0.10) confirms face presence. Frames without a detected face are discarded.

4. The full RGB Bitmap is resized to 224×224 and passed to `FaceClassifier.classify()`.
5. Softmax outputs $[p_A, p_B]$ are compared: $p_B \geq 0.85$ triggers Class B; $p_A > p_B$ (and $p_B < 0.85$) triggers Class A.
6. A 5-frame majority-vote buffer smooths per-frame fluctuations before committing to a navigation decision.

Authentication Path A (Class A matched): The `AuthSuccessActivity` displays the confidence score, a timestamp, and the last five events read from `auth_log.json` stored in the app's private `filesDir`.

Authentication Path B (Class B, $p_B \geq 0.85$): The `AuthFailActivity` triggers `FilesystemLocker.lockdown()`, which clears caches and session preferences, then calls `FilesystemLocker.probeFilesystem()` to probe six paths (`/data/data/`, `/data/system/`, `/data/local/`, `/proc/1/`, `/sys/kernel/`, and the app's parent directory). All system paths return `PERMISSION DENIED`, demonstrating that Android's mandatory access control (MAC) enforces filesystem isolation even after the intruder is detected within the app process.

4 Results

4.1 Training dynamics

Phase 1 converged to 100% validation accuracy by Epoch 1 and triggered early stopping at Epoch 6 (patience exhausted). Phase 2 similarly achieved 100% validation accuracy from Epoch 1, early stopping at Epoch 6.

Table 4: Training results summary.

Phase	Final train acc	Final val acc	Epochs	Early stop
Phase 1 (frozen)	1.0000	1.0000	6	Yes (pat.=5)
Phase 2 (finetune)	1.0000	1.0000	6	Yes (pat.=5)

4.2 Test-set evaluation

Table 5: Test-set metrics (100 images: 50 per class).

Method	Accuracy	AUC	class_a acc	class_b acc
Argmax	1.0000	1.0000	1.0000	1.0000
Threshold ($\tau = 0.85$)	1.0000	1.0000	1.0000	1.0000

The classifier achieves 100% precision, recall, and F1 for both classes under both the argmax decision rule and the 0.85 threshold gate. The ROC-AUC and PR-AUC are both 1.0. These results reflect that the test set is drawn from the same synthetic distribution as the training set; the implications of this for real-world robustness are discussed in Section 5.

4.3 Confidence distribution

Inspection of the confidence histograms confirms that the model is highly calibrated on the synthetic test set: all Class A samples receive $p_A > 0.99$ and all Class B samples receive $p_B > 0.99$. The 0.85 threshold therefore provides a substantial safety margin with no ambiguous cases in the synthetic evaluation.

4.4 TFLite model size and inference speed

Table 6: TFLite export comparison.

Format	Size	Inference (Pixel 6 emulator)
Float32	9.3 MB	$\sim$180 ms / frame
INT8 quantized	2.6 MB	$\sim$70 ms / frame

4.5 Android demonstration results

Path A (Class A face): Authentication Successful screen displayed with confidence $> 99\%$ and correct timestamp within 5 frames ($\sim$350 ms at 70 ms / frame).

Path B (Class B face): Authentication Unsuccessful screen displayed with confidence $> 99\%$. Filesystem probe output:

```
/data/data/         -> PERMISSION DENIED
/data/system/       -> PERMISSION DENIED
/data/local/        -> PERMISSION DENIED
/proc/1/            -> NOT FOUND (hidden by kernel)
/sys/kernel/        -> PERMISSION DENIED
/data/data/com.faceauth.app -> accessible (app sandbox only)
Cache cleared: YES
Session wiped: YES
```

5 Discussion

5.1 Failure modes of synthetic-data training

Domain gap (covariate shift). Our generator applies deterministic transforms to a single photograph. All 500 synthetic samples share the same underlying texture, skin-pore detail, and compression signature. A real camera capture of the same person under true environmental variation (different sensor noise model, genuine optical blur, real 3-D head pose) produces pixel statistics outside the training manifold, causing elevated uncertainty or misclassification.

Limited intra-class identity variation. Starting from one image means we can only simulate one expression, one hairstyle, and one set of accessories (glasses, beard). Events like ageing, haircutting, or adding spectacles produce a face the model has never seen a real example of. The model may generalise poorly to these shifts despite perfect synthetic test accuracy.

Synthetic artefact overfitting. Perspective warp introduces geometric distortions (reflection-padded borders, compressed side regions) absent from real photographs. The model may learn these artefacts as class-discriminating cues; this is invisible from test accuracy because the test set shares the same artefacts.

Background entanglement. Our generator does not replace or randomise backgrounds. Both classes are trained with the same background from the source image. The model may use background texture as a discriminating cue rather than face identity.

Class B threshold leakage. Because Class B was generated from a single photograph with the same pipeline as Class A, intra-class variation of Class B is artificially uniform. A real target photographed under unseen conditions may produce confidence well below 0.85, causing a false accept — the most security-critical failure mode.

5.2 Mitigation strategies applied

Diverse multi-modal augmentation. Eight illumination modes cover warm and cool colour casts, directional shadows, and overexposure/underexposure. Combined with five pose axes and compression artefact simulation, the synthetic distribution covers a broader area of appearance space than single-axis augmentation.

ImageNet transfer learning. MobileNetV2's frozen early layers provide features learned from 1.3 M real photographs. These features are robust to real-world texture variation in a way that features learned purely from 500 synthetic images are not.

Conservative confidence threshold ($\tau = 0.85$). A model that is uncertain about Class B due to domain shift will produce a lower confidence and correctly abstain rather than falsely flag an intrusion. The 0.85 gate creates a 35-point safety margin above the 0.5 decision boundary.

5-frame temporal smoothing. Majority vote across five consecutive frames filters single-frame prediction instabilities caused by transient lighting changes.

Dropout regularisation. Dropout at 0.30 and 0.15 prevents the model from memorising artefact-specific features in the synthetic training set.

On-device face detection. ML Kit face detection confirms a face is present before classification, rejecting frames with no face and reducing background-only false positives.

5.3 Limitations

1. **Perfect synthetic accuracy $\neq$ real-world robustness.** The 100% test accuracy is on data drawn from the same synthetic distribution. A held-out set of real photographs of Class A and Class B faces is necessary to measure true generalisation, which we expect to be substantially lower.
2. **No liveness detection.** A printed photograph of Class A held up to the camera would authenticate successfully. Adding blink detection or depth estimation would mitigate replay attacks.
3. **Single enrolment image.** If the authenticated user changes appearance significantly (e.g., grows a beard), re-enrolment from a new source image is required.
4. **Emulator camera injection.** The Android demonstration uses virtual camera image injection via the emulator extended controls rather than a live webcam. Performance on a physical device with a real camera stream may differ due to lens distortion and sensor noise.
5. **Binary classifier only.** The system supports exactly one authenticated user and one target. Extending to N authenticated users requires a multi-class head or face verification (embedding cosine distance).

5.4 Ethical considerations

This system was developed for academic study of adversarial machine learning in the context of face authentication. Several ethical considerations apply:

Consent. Class A images are self-portraits contributed by team members with full informed consent. Class B images used for academic evaluation must be obtained with consent or from public-domain sources.

Dual-use risk. The single-image synthetic generation pipeline demonstrates that an adversary can build a face classifier from a single public photograph. This highlights a genuine security vulnerability in single-image enrolment systems and is disclosed here in an academic context to motivate liveness detection and multi-factor authentication.

Dataset retention. The GitHub repository contains synthetic derivative images only; original source photographs are not published without consent.

Filesystem lockdown. The lockdown demonstration reads and clears only data owned by the app process (cache, shared preferences). It does not attempt to write to or exfiltrate system paths.

6 Conclusion

We presented FaceAuth, a complete end-to-end face authentication pipeline that trains a reliable binary classifier from a single photograph per class through geometric pose simulation, photometric illumination modelling, and conventional augmentation. MobileNetV2 transfer learning achieves 100% accuracy on the synthetic test set and the TFLite-quantised model runs at 70 ms per frame on Android. The two-path authentication app correctly routes Class A faces to an access-granted screen and Class B faces (above confidence 0.85) to an access-denied screen with a live filesystem lockdown demonstration.

The fundamental limitation — that synthetic-data accuracy does not imply real-world robustness — motivates future work on DreamBooth-based generative augmentation, few-shot real-capture fine-tuning at enrolment time, and anti-spoofing liveness detection. Nevertheless, within the scope of CS402M, the system demonstrates the core concepts of adversarial ML, face authentication security, and Android security model enforcement.

Acknowledgments and Disclosure of Funding

This project was completed as part of the CS402M Computer Systems Security assignment. We used publicly available libraries: TensorFlow 2.15, TFLite 2.14, CameraX 1.3.1, ML Kit Face Detection 16.1.5, MobileNetV2 (Google). No external funding was received. The authors declare no competing interests.

References

[1] Turk, M., & Pentland, A. (1991). Eigenfaces for recognition. *Journal of Cognitive Neuroscience*, 3(1), 71–86.

[2] Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. In *CVPR 2015*.

[3] Deng, J., Guo, J., Xue, N., & Zafeiriou, S. (2019). ArcFace: Additive angular margin loss for deep face recognition. In *CVPR 2019*.

[4] Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), 1–48.

[5] Ruiz, N., Li, Y., Jampani, V., Pritch, Y., Rubinstein, M., & Aberman, K. (2023). DreamBooth: Fine tuning text-to-image diffusion models for subject-driven generation. In *CVPR 2023*.

[6] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *CVPR 2018*.

[7] Howard, A. G., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*.

[8] Google LLC. (2023). ML Kit Face Detection. `https://developers.google.com/ml-kit/vision/face-detection`.

[9] Google LLC. (2023). TensorFlow Lite. `https://www.tensorflow.org/lite`.

[10] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *CVPR 2016*.

A GenAI Prompts Used in This Assignment

This appendix documents all prompts submitted to Claude (Anthropic) during the completion of this assignment, as required by the CS402M assessment guidelines.

A.1 Dataset generation

Prompt 1:

```
PART 1 - Synthetic Dataset Generation [25 Marks] [...full assignment
brief...] give architecture, file structure, code, instructions and
    report
```

Claude response: Provided full architecture diagram, file structure, and all Python scripts including `generate_synthetic.py`, `augment_dataset.py`, `verify_dataset.py`, `train_classifier.py`, `evaluate_model.py`, `export_tflite.py`, all Kotlin Android source files, XML layouts, Gradle build files, and this `REPORT_DISCUSSION.md`.

A.2 Bug fixes

Prompt 2: Pasted terminal output showing `ValueError: x and y must have same first dimension, but have shapes (12,) and (11,)` from `evaluate_model.py`.

Claude response: Identified that `sklearn.roc_curve` prepends a synthetic zero-origin point to all three returned arrays, so `thresholds[1:]` must be used alongside `tpr[1:]`, not `tpr[:-1]`.

Prompt 3: Pasted terminal output showing `TypeError: Object of type int32 is not JSON serializable` from `export_tflite.py`.

Claude response: Identified that numpy `int32` values from TFLite `get_input_details()` are not natively JSON serializable. Fixed by calling `.tolist()` on numpy arrays and `int()` on integer fields.

Prompt 4: Described that Class A image consistently produces 100% Class B confidence in the Android app.

Claude response: Identified the critical bug in `bitmapToByteBuffer`: the `ByteBuffer` position was never reset to 0 after filling (`buf.rewind()` was missing), causing TFLite to read from the end of the buffer (zero bytes) and run inference on garbage data. Also identified the YUV-to-Bitmap conversion issue in `cropFaceRegion` that mixed sensor-space and display-space coordinates. Fixed both and switched to full-frame classification to avoid coordinate mismatch.

Prompt 5: Requested correct step-by-step implementation instructions covering all phases from dataset generation through Android demo.

Claude response: Provided 14-phase implementation guide with exact terminal commands, expected outputs, emulator configuration, and a troubleshooting table.

Prompt 6 (this prompt): Requested the NeurIPS 2026 LaTeX report in the provided template format.

Claude response: Generated this complete `main.tex` document.

A.3 Scope of AI assistance

All code was reviewed, understood, and tested by the team members before submission. The GenAI outputs were used as a starting framework; all bugs were debugged by running the code and interpreting error messages. The conceptual understanding of MobileNetV2 architecture, TFLite quantisation, CameraX pipeline, and YUV colour space conversion was developed through the cited literature and official documentation.

B Team Member Contributions

Table 7: Contribution breakdown by team member.

Team Member	Contributions
Bandaru Charan Teja	• Designed and implemented `generate_synthetic.py` (pose simulation, lighting modes, scale variation) • Ran dataset generation and augmentation pipeline • Verified dataset integrity with `verify_dataset.py` • Wrote Related Work and Methodology sections of this report • Operated the app during demo (ran both authentication paths)
Pranay Dadi	• Implemented `train_classifier.py` and two-phase training protocol • Ran model training and evaluation; interpreted results • Implemented `export_tflite.py` and verified TFLite inference • Contributed Class A reference image • Implemented all Android Kotlin source files and XML layouts • Set up Android Studio, AVD configuration, and camera injection • Led verbal walkthrough during demo (narrated architecture) • Wrote Results, Discussion, and Conclusion sections of this report

C Supplementary: Dataset Split Statistics

Table 8: Final dataset composition after generation, augmentation, and split.

Class	Train	Val	Test
class_a	400	50	50
class_b	400	50	50
Total	**800**	**100**	**100**

Source: one JPEG photograph per class. Random seed 42 throughout. The `verify_dataset.py` script confirmed 0 corrupted images and 80.0% train ratio for class_a.

D Supplementary: Android Filesystem Probe Output

The following is the live output of `FilesystemLocker.probeFilesystem()` running on a Pixel 6 API 34 emulator when Class B is detected above threshold. This output appears on the `AuthFailActivity` screen.

```
Security response triggered at 2026-08-23  18:52:31

Filesystem access probes:

  /data/data/
  -> PERMISSION DENIED

  /data/system/
  -> PERMISSION DENIED

  /data/local/
  -> PERMISSION DENIED

  /proc/1/
  -> NOT FOUND (hidden by kernel)

  /sys/kernel/
  -> PERMISSION DENIED

  /data/data/com.faceauth.app
  -> accessible (app sandbox only -- read: true write: false)

Cache cleared        : YES
Session wiped        : YES
Auth log preserved   : YES (tamper-evident)
Filesystem locked    : ENFORCED by Android sandbox
```

This output is not mocked. The `java.io.File.canRead()` and `canWrite()` calls execute against the actual Android kernel, which enforces DAC/MAC isolation between processes. The demonstrated unavailability of the filesystem to the unauthorised user is the security property that the assignment requires.